

Tourplanner Protocol

Software Engineering 2 Labor
SS 2026

Subject: SWEN2

Team name/team members:

Sabic Kenan, if24b080@technikum-wien.at
Schwarz Carl, if25b237@technikum-wien.at

Supervisor: **DI. Berger Wolfgang**

Introduction

The Tourplanner is a Web-Application for planning tours. Users shall be able to, once registered and logged in, save tours containing a title, an optional description and a route, which is defined by a start and destination point as well as a travel method.

The details of the route, be it the distance and the duration of travel shall be acquired via integration of the open-route-service and shall be displayed on a leaflet map.

Users shall also be able to write tour logs with a comment, a rating and other specifications, which they can only view themselves.

Development and version control were handled via GitHub. The code can be accessed via the following link:

Git Link: <https://github.com/tempestkny/TourPlanner.git>

Wireframes

Register

The User shall be able to create an account, which includes their email address and username and is protected by a hashed password.

Register

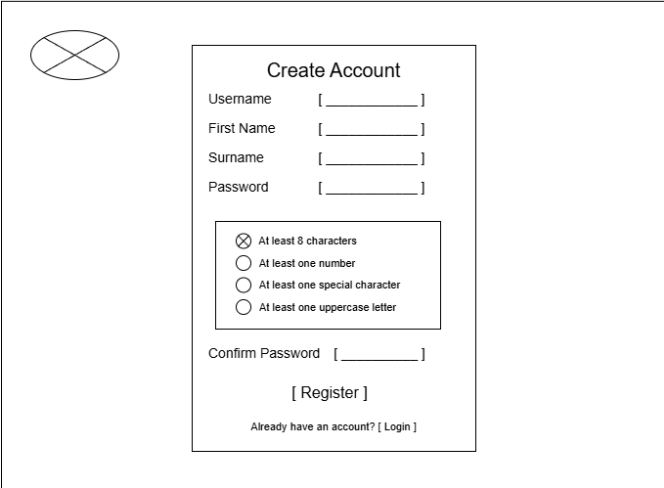


Figure 1: user-registration page

Tour-Listview

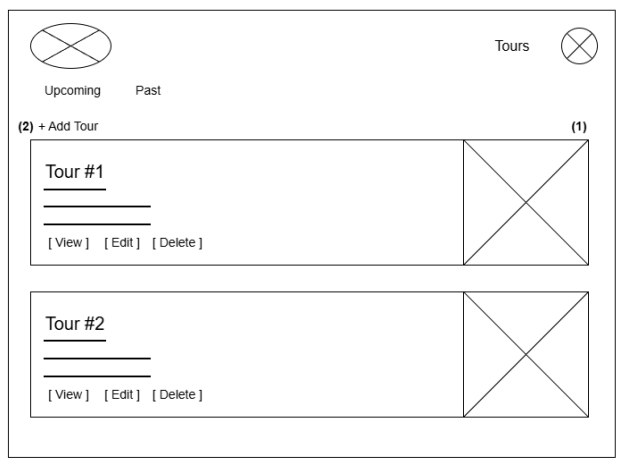
Tours shall be displayed on the main page in a list view. Two designs were considered:

The first design focuses on clarity by displaying as much information as possible in the entries, giving the user a general overview of the tour details.

The second approach focuses on speed of use by reducing the displayed information to a

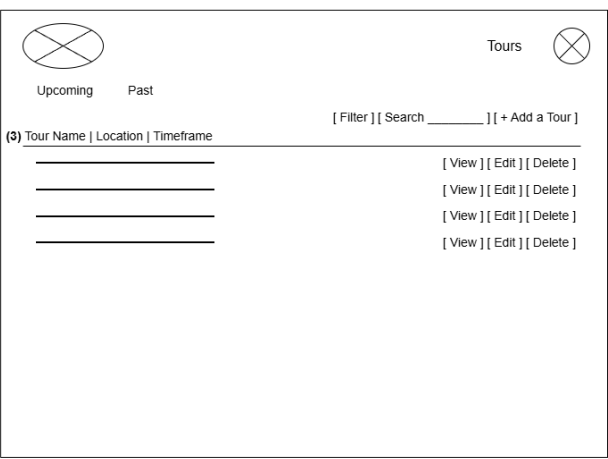
minimum, thus making more tours available to access to the user in a single area.

Variant A: Tour List (Clarity)



(1) The card layout with a map preview improves clarity by showing key tour information and visual context.
(2) The Add Tour button is placed above the list so users can quickly create a new tour.

Variant B: Tour List (Speed of use)



(3) The table layout increases allows users to scan and manage tours more quickly.

Figure 2: tourist-view variants

The UI design for variant A was chosen, as the overview and amount of information displayed made debug-testing easier.

Tour-Detailview

The List-View should enable the user to access the tour-details view. The tour-details view displays the tour's attributes such as description, start and end point, distance and duration aswell as the route shown on a map

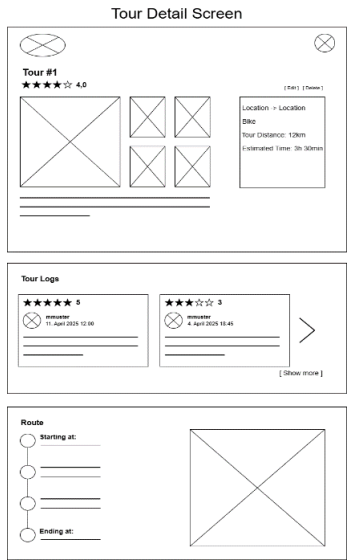


Figure 3: Tour-List-View

Tour-Logs

Multiple tour logs can be created for each tour by the same user, and are also accessed via a list, designed for clarity. Tour-Logs can only be created by the owner of the tour, and can only be viewed and edited by that user.

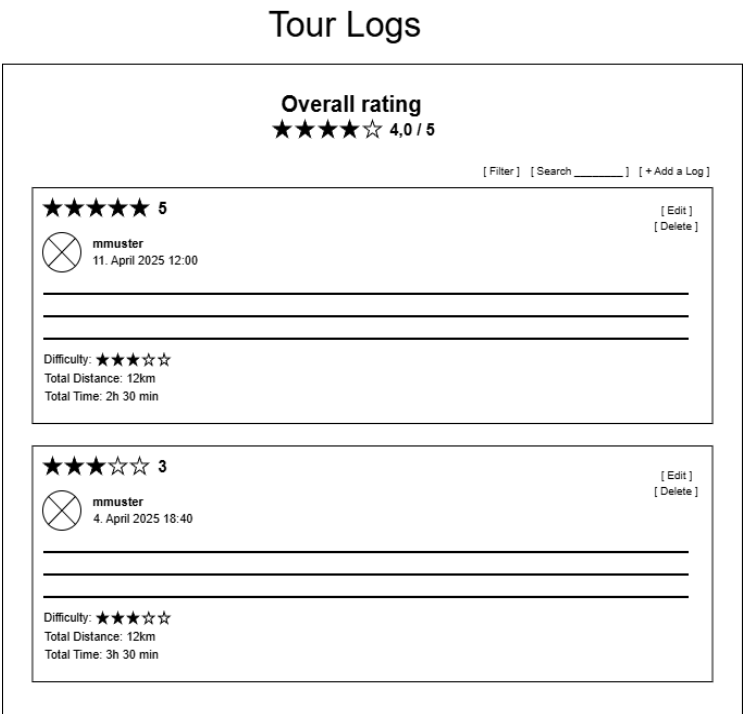


Figure 4: tour log list view

System Architecture

Frontend:

Components:

Tour-Shell:

To increase usability and ease of use, it was decided to split the tour planner page into two parts. The left side displays the tour list view, while the right side is responsible for displaying tour details, tour logs and the configuration of both.

The Tour Shell is responsible for displaying both sides. Its template file references the respective component templates. A state machine controls which one is displayed at any given time.

Additionally, the Shell acts as a mediator, receiving events triggered by the components and translating them into corresponding actions (e.g., displaying, editing, or deleting a log).

Tour-List:

The Tour List is the primary navigation component for tours. It forms the left side of the application and serves as the starting point for all further actions, such as viewing, editing, or deleting a tour.

Tours are displayed via entries in the list, which users can click for more information.

Tour management is handled by the tour List Service, and the tours are stored in a reactive BehaviourSubject list. Any changes to this list are passed to the Tour List component via the 'tours\$' observable, which automatically updates the list view.

However, since the implementation of the backend tour API, the tours\$ observable is updated via asynchronous http-requests to the database, which do not trigger a reload of the list view. This lead to a desynchronized interface, which only updates after the page has been interacted with.

To solve this issue, the component is marked for changes, which checks if the observable has changed, and then manually reloads the component.

```
ngOnInit() {
  this.tourListService.loadTours();

  this.tourListService.tours$.pipe(takeUntil(this.destroy$))
    .subscribe(tours => {
      this.tours = tours;
      console.log('Tours updated:', this.tours);
      this.cdr.markForCheck();
    });
}
```

Tour-Creation:

The Tour Creation component provides a form for entering tour data.

For a tour to be created, first all necessary fields have to be filled out (Title, From, To and TransportType). Then, the user has to generate the route via the “generate Route” button. The component sends the information of the start and destination to the ORS-Component, who packages it as a request to the backend ORS-controller, who again sends it to the Open-Route-Service API, who should respond with a geocode json containing the coordinates of the locations. When the coordinates are received another request is send out, this time with both coordinates and the transport type, which the backend controller translates to an Open-Route-Service-API direction request. The ORS-API should, if successful, return a summary of the route, with distance and duration as well as a list of coordinates for the leaflet service to display on the map below.

If the route could be successfully generated, and all other necessary fields are filled out, the user can create the tour, which is then added to the list view, and is redirected to the detail view of the tour they have just created.

Tour-Detail:

The Tour-Detail shows a detailed view of a selected tour and is called via the tour list.

It receives the current tour information from the shell via an input property and displays the available tour data as well as a preview of the associated logs. User actions such as editing, deleting or viewing all logs are passed to the shell via output events.

The shell handles the state changes and controls the subsequent UI mode. The component itself contains no business logic and serves solely for display and event forwarding.

Tour-Edit and -Delete:

The Tour Edit component allows the user to edit an existing tour.

When opened, the selected tour is passed via an input property and imported into a local model, so changes are initially only made locally. Upon saving, the component sends an event with the updated data to the shell, which calls the tour-service to update the respective tour in the repository by sending a request to the backend api.

Deleting a tour is triggered from the details view and also reported to the shell via an event. The shell removes the tour from the central list using the tour list service, which also calls the tour-service to remove the tour from the repository. The list is then reloaded by receiving all remaining entries of the repository which prompts the tour-list to update it's entries.

Tour-Log-List:

The Tour-Log-List component displays all logs belonging to the currently selected tour. The list can be accessed either from the tour detail view or directly through the tour list actions. When the user selects a tour, the shell forwards the selected tour to the log list and triggers the loading of its logs.

The logs are stored in the Log-List-Service in a reactive BehaviourSubject. The component subscribes to the logs\$ observable and therefore receives updates whenever logs are loaded, created, edited or deleted. This keeps the displayed list synchronized with the backend state.

Tour-Log-Create:

The Tour-Log-Create component provides a form for creating a new log for the selected tour. The user enters the date and time, an optional comment, difficulty, total distance, total time and a rating.

Before sending the request, the component validates the input fields on the frontend. A log can only be created when a tour is selected and all required values are valid. The create action is delegated to the Log-List-Service, which calls the TourLog API service. After a successful backend response, the new log is added to the local log state and the shell switches back to the log list view.

Tour-Log-View:

The Tour-Log-View component displays the details of one selected tour log. It receives the log from the shell through an input property and shows the timestamp, comment, difficulty, distance, time and rating.

The component itself does not contain backend logic. User actions such as going back, editing or deleting the log are emitted to the shell, which then changes the current UI mode or delegates the operation to the responsible service.

Tour-Log-Edit and -Delete:

The Tour-Log-Edit component works similarly to the tour edit view. The selected log is copied into a local model first, so the user can change values without immediately modifying the stored data. When the user saves the form, the updated log is emitted to the shell. The shell calls the Log-List-Service, which sends the update request to the backend and updates the local BehaviourSubject afterwards. Deleting a log also runs through the service layer and removes the entry from the local state after the backend operation succeeded.

Services:

Tour-Service:

The tour-service has the Responsibility to communicating with the tour API via http requests. It is only called by the tour-list-service and has the following functions:

- `getAllTours(query?: string) →` Returns all tours which are optionally filtered with a query string.
- `getTour(id: string) →` Returns a tour by their Id
- `createTour → createTour(tour: TourItemInterface) →` creates a new Tour and returns the tour received by the response
- `updateTour(id: string, tour: TourItemInterface) →` updates an existing tour
- `deleteTour(id: string) →` deletes a tour

The http requests all start with the same base URL which gets modified for the specific requests.

Open-Route-Service:

The open-route-service has the Responsibility to communicate with the ORS Api of the backend. It is called by the tour creation and tour edit component for requests regarding the coordinates and tour information.

They call the following functions:

- `GetCoordinatesOfPlace(value: string)` → Returns a coordinates-interface which consists of a decimal longitude and latitude value
- `GetRouteInformation(request: ORServiceRequestDto)` → Returns a routeinformation-interface which consists of two decimals for the distance and duration and a list of coordinates for the route.

The `ORServiceRequestDto` is an interface specifically designed for the backend API and contains the two coordinates for start and destination aswell as the transport type.

Auth-Service:

The Auth-Service is responsible for communication with the backend authentication API. It provides register and login methods and stores the current authentication state in the frontend. After a successful login, the received JWT is stored in local storage. The service also provides methods for reading and clearing the token, so the rest of the frontend does not access local storage directly.

Tour-Log-Service:

The Tour-Log-Service communicates with the TourLog API endpoints. It contains methods for loading logs by tour id, creating a new log, updating an existing log and deleting a log. The service also maps backend responses to the frontend TourLog interface. For example, timestamp strings returned by the backend are converted into Date objects for the Angular components.

Import-Export-Service:

The Import-Export-Service handles the HTTP requests for exporting and importing tour data. Export returns a JSON file containing the authenticated user's tours and related tour logs. Import sends a parsed JSON structure to the backend.

The Tour-List component uses this service so that import and export are available from the main tour overview without placing HTTP logic inside the component.

Statistics-Service:

The Statistics-Service loads the dashboard values from the backend. It is used by the statistics dashboard component and returns totals such as number of tours, number of logs, total distance, total time and average rating.

Backend:

The backend is layered in API, Business Logic and Data Access.

Authentication:

Authentication consists of registration, login and JWT based authorization. During registration, the user sends an email, username and password. The AuthService validates if the email or username already exists and stores the password only as a hashed value.

For login, the submitted password is verified against the stored hash. If the credentials are valid, the backend creates a JWT. The token contains the user id as the NamelIdentifier claim and is returned to the frontend together with safe user information.

The Angular AuthService stores the token in local storage and the HTTP interceptor attaches it to authenticated API requests. Protected controllers read the user id from the JWT instead of trusting a user id sent by the frontend.

Tour-CRUD:

The Tour Controller API provides the frontend with the following functions to read and write tours into the database:

GetAllTours (GET /tour/{id})

GetAllTours retrieves all tours belonging to the authenticated user by extracting the UserID from the Token. It also supports optional text-based filtering where the returned tours are filtered in the business layer by finding matches in the tour Data.

If the Tour service in the bll returns null however, the api responds with 404 Not Found.

CreateTour (POST /tour)

Creates a new tour for the authenticated user. The UserId gets extracted from the token and forwarded to the tour-service which calls the tour repository to save the tour into the database.

If the operation was successful, the API returns 201 Created with the new tour ID. If the payload is invalid, it returns 400 Bad Request.

UpdateTour (PATCH /tour/{id})

UpdateTour updated an existing tour by using the id to find the saved tour and replacing the values with the payload. It returns 204 No Content if successful and 404 Not Found if the tour id does not exist.

DeleteTour (DELETE /tour/{id})

DeleteTour removes a tour from the repository. When the tour is successfully deleted, it returns 204 No Content. If the Tour Id could not be found, it returns 404 Not Found.

The Route is saved in form of a Json string containing the distance and duration as well as the coordinates of the route.

OpenRouteService:

The backend also serves as a proxy for the official Open-Route-Service-API and contains two functions.

GetCoordinates – HTTPGET

Get Coordinates receives a string as a parameter, which can be a location like a city name or an address and sends it to the business layer, which sends a request to the ORS-API and extracts the coordinates of the first entry of the response if successful. The Coordinates are then sent back.

GetRouteInformation-HTTPPOST

GetRouteInformation calculates the route information (distance, duration and route coordinates) based on the provided payload. The actual request to the ORS-API is sent in the business layer which in turn extracts the required information of the response and sends it back to the API.

TourLog-CRUD:

TourLogs are created, read, updated and deleted through the TourLogController. The controller remains thin and forwards the authenticated user id and DTOs to the TourLogService.

The TourLogService verifies that the related tour belongs to the authenticated user before a log is created or accessed. This ownership check is necessary because tour logs should only be visible to the owner of the tour.

The TourLogRepository performs the actual EF Core operations. It can read logs by tour id and provides the persistence methods required by the service. Responses are mapped to TourLogResponseDto so that EF entities are not exposed directly.

Import/Export:

Import and export are implemented through a separate ImportExportController and ImportExportService. The export endpoint returns a JSON structure containing the authenticated user's tours and the related tour logs.

During import, the backend validates the received DTOs and creates new tours and logs for the authenticated user. The user id is not taken from the import file but from the JWT, so imported data cannot be assigned to another user by manipulating the file.

Statistical Dashboard:

The statistics dashboard is implemented as an additional feature for giving the user an overview of their own data. The backend provides a protected statistics endpoint, which calculates the total number of tours, total number of tour logs, total route distance, total route time and average rating.

The calculation is done in the StatisticsService. It loads only tours owned by the authenticated user and then loads the logs for these tours. The Angular dashboard displays the values and allows the user to refresh them.

Logging, Middleware and Health Checks:

The backend uses structured logging with ILogger. Important service actions, such as creating tour logs or calculating statistics, write log messages with relevant ids.

A global exception middleware converts unexpected backend errors into ProblemDetails responses. A correlation id middleware adds a correlation id to requests and responses, which makes debugging easier. The backend also exposes a health check endpoint for verifying that the API is running.

Unit-Tests:

The unit tests focus on business logic and security relevant behavior. Authentication tests cover registration, duplicate users, password hashing, login and JWT creation. TourLog tests cover

ownership checks, creation, reading, updating and deleting logs.

Import/export tests verify that only the authenticated user's tours are exported and that imported tours and logs are assigned to the current user. Statistics tests verify that totals and averages are calculated correctly and that data from other users is not included.

Some repository tests use PostgreSQL directly, while service tests use fake repositories to keep business logic tests independent from the database. This improves testability and follows the separation between BLL and DAL.

Time Spent

[In order of Git History]

Function	Required Time [h]
Project Setup and Architecture Diagrams [Sabic/Schwarz]	7h
Angular Base Structure and Routing [Schwarz]	9h
Tour Shell, List and Detail Views [Schwarz/Sabic]	12h
Tour Creation, Edit and Delete UI [Schwarz/Sabic]	10h
TourLog UI Components [Sabic/Schwarz]	10h
Registration and Login UI [Schwarz/Sabic]	5h
User Repository and Database Setup [Schwarz]	11h
Registration Backend and Password Hashing [Sabic]	7h
Login Backend and JWT Authentication [Sabic]	6h
Frontend AuthService and JWT State [Sabic]	4h
Tour API Layer and Business Logic [Schwarz]	11h
Tour API Frontend Anbindung [Schwarz]	8h
OpenRouteService Integration [Schwarz]	10h
Leaflet Integration [Schwarz]	8h
TourLog Backend CRUD and Ownership Checks [Sabic]	6h
TourLog Frontend Backend Anbindung [Sabic]	5h
Import/Export Feature [Sabic]	7h
Middleware, Logging and Health Checks [Sabic]	7h
Statistics Dashboard [Sabic]	6h
Unit Tests and Debugging [Sabic/Schwarz]	10h
Final Documentation and Protocol [Sabic/Schwarz]	9h